IT草根

Lucene 6.0 实战（4）–文本分析器

2016-05-23 | 2017-11-20 | Programming Notes | 1512

Analyzer简介

在Lucene的org.apache.lucene.analysis模块中提供了顶层的抽象类Analyzer，Analyzer主要是用来构建TokenStreams，如果想实现自定义的Analyzer，必须覆写createComponents(String)方法，并定义自己的TokenStreamComponents。

为什么要有Analyzer呢？对于Lucene而言，不管是索引还是检索，都是针对纯文本而言，对于纯文本的来源可以是PDF，Word，Excel，PPT，HTML等，Lucene对此并不关心，只要保证传递给Lucene的是纯文本即可。

而通常情况下，对于大量的文本，用户在检索的时候不可能全部输入，例如有文本“Hello World，I’am javaer!”，用户在检索的时候可能只是输入了“Hello”，这里就需要Lucene索引该文本的时候预先对文本进行切分，这样在检索的时候才能将词源和文本对应起来。

Lucene常用分析器整理如下

分析器	说明
WhitespaceAnalyzer	根据空格拆分语汇单元
SimpleAnalyzer	根据非字母拆分文本，并将其转换为小写形式
StopAnalyzer	根据非字母拆分文本，然后小写化，再移除停用词
KeywordAnalyzer	将整个文本作为一个单一语汇单元处理
StandardAnalyzer	根据Unicode文本分割算法，具体算法参考Unicode Standard Annex #29，然后将文本转化为小写，并移除英文停用词
martChineseAnalyzer	SmartChineseAnalyzer 是一个智能中文分词模块，能够利用概率对汉语句子进行最优切分，并内嵌英文 tokenizer，能有效处理中英文混合的文本内容。它的原理基于自然语言处理领域的隐马尔科夫模型（HMM），利用大量语料库的训练来统计汉语词汇的词频和跳转概率，从而根据这些统计结果对整个汉语句子计算最似然（likelihood）的切分。因为智能分词需要词典来保存词汇的统计值，SmartChineseAnalyzer 的运行需要指定词典位置，如何指定词典位置请参考org.apache.lucene.analysis.cn.smart.AnalyzerProfile。SmartChineseAnalyzer 的算法和语料库词典来自于ICTCLAS

猜你喜欢

分析器	说明
CJKAnalyzer	CJK表示中日韩，目的是要把分别来自中文、日文、韩文、越文中，本质、意义相同、形状一样或稍异的表意文字（主要为汉字，但也有仿汉字如日本国字、韩国独有汉字、越南的喃字）在ISO 10646 及 Unicode 标准内赋予相同编码。对于中文是交叉双字分割，二元分词法

Analyzer部分子类分词示例

选取了六个实现类，并分别输出它们对英文、中文、特殊符号及邮箱等的切分效果。

```
public class AnalyzerDemo {
    private static final String[] examples = {"The quick brown 1234 fox jumped over the lazy dog!", "XY&Z
            "北京市北京大学"};
    private static final Analyzer[] ANALYZERS = new Analyzer[]{new WhitespaceAnalyzer(), new SimpleAnalyz
            StandardAnalyzer(), new CJKAnalyzer(), new SmartChineseAnalyzer()};

    @Test
    public void testAnalyzer() throws IOException {
        for (int i = 0; i < ANALYZERS.length; i++) {
            String simpleName = ANALYZERS[i].getClass().getSimpleName();
            for (int j = 0; j < examples.length; j++) {
                TokenStream contents = ANALYZERS[i].tokenStream("contents", examples[j]);
                //TokenStream contents = ANALYZERS[i].tokenStream("contents", new StringReader(examples[
                OffsetAttribute offsetAttribute = contents.addAttribute(OffsetAttribute.class);
                TypeAttribute typeAttribute = contents.addAttribute(TypeAttribute.class);
                contents.reset();
                System.out.println(simpleName + " analyzing : " + examples[j]);
                while (contents.incrementToken()) {
                    String s1 = offsetAttribute.toString();
                    int i1 = offsetAttribute.startOffset();//起始偏移量
                    int i2 = offsetAttribute.endOffset();//结束偏移量
                    System.out.print(s1 + "[" + i1 + "," + i2 + ":" + typeAttribute.type() + "]" + " ");
                }
                contents.end();
                contents.close();
                System.out.println();
            }
        }
    }
}
```

输出结果如下

```
WhitespaceAnalyzer analyzing : The quick brown 1234 fox jumped over the lazy dog!
The[0,3:word] quick[4,9:word] brown[10,15:word] 1234[16,20:word] fox[21,24:word] jumped[25,31:word] over
WhitespaceAnalyzer analyzing : XY&Z 15.6 Corporation - xyz@example.com
XY&Z[0,4:word] 15.6[5,9:word] Corporation[10,21:word] -[22,23:word] xyz@example.com[24,39:word]
WhitespaceAnalyzer analyzing : 北京市北京大学
北京市北京大学[0,7:word]
SimpleAnalyzer analyzing : The quick brown 1234 fox jumped over the lazy dog!
the[0,3:word] quick[4,9:word] brown[10,15:word] fox[21,24:word] jumped[25,31:word] over[32,36:word] the[
SimpleAnalyzer analyzing : XY&Z 15.6 Corporation - xyz@example.com
xy[0,2:word] z[3,4:word] corporation[10,21:word] xyz[24,27:word] example[28,35:word] com[36,39:word]
SimpleAnalyzer analyzing : 北京市北京大学
北京市北京大学[0,7:word]
StopAnalyzer analyzing : The quick brown 1234 fox jumped over the lazy dog!
quick[4,9:word] brown[10,15:word] fox[21,24:word] jumped[25,31:word] over[32,36:word] lazy[41,45:word] d
```

```
StopAnalyzer analyzing : XY&Z 15.6 Corporation - xyz@example.com
xy[0,2:word] z[3,4:word] corporation[10,21:word] xyz[24,27:word] example[28,35:word] com[36,39:word]
StopAnalyzer analyzing : 北京市北京大学
北京市北京大学[0,7:word]
StandardAnalyzer analyzing : The quick brown 1234 fox jumped over the lazy dog!
quick[4,9:<ALPHANUM>] brown[10,15:<ALPHANUM>] 1234[16,20:<NUM>] fox[21,24:<ALPHANUM>] jumped[25,31:<ALPH
StandardAnalyzer analyzing : XY&Z 15.6 Corporation - xyz@example.com
xy[0,2:<ALPHANUM>] z[3,4:<ALPHANUM>] 15.6[5,9:<NUM>] corporation[10,21:<ALPHANUM>] xyz[24,27:<ALPHANUM>]
StandardAnalyzer analyzing : 北京市北京大学
北[0,1:<IDEOGRAPHIC>] 京[1,2:<IDEOGRAPHIC>] 市[2,3:<IDEOGRAPHIC>] 北[3,4:<IDEOGRAPHIC>] 京[4,5:<IDEOGRAPHI
CJKAnalyzer analyzing : The quick brown 1234 fox jumped over the lazy dog!
quick[4,9:<ALPHANUM>] brown[10,15:<ALPHANUM>] 1234[16,20:<NUM>] fox[21,24:<ALPHANUM>] jumped[25,31:<ALPH
CJKAnalyzer analyzing : XY&Z 15.6 Corporation - xyz@example.com
xy[0,2:<ALPHANUM>] z[3,4:<ALPHANUM>] 15.6[5,9:<NUM>] corporation[10,21:<ALPHANUM>] xyz[24,27:<ALPHANUM>]
CJKAnalyzer analyzing : 北京市北京大学
北京[0,2:<DOUBLE>] 京市[1,3:<DOUBLE>] 市北[2,4:<DOUBLE>] 北京[3,5:<DOUBLE>] 京大[4,6:<DOUBLE>] 大学[5,7:<DOU
SmartChineseAnalyzer analyzing : The quick brown 1234 fox jumped over the lazy dog!
the[0,3:word] quick[4,9:word] brown[10,15:word] 1234[16,20:word] fox[21,24:word] jump[25,31:word] over[3
SmartChineseAnalyzer analyzing : XY&Z 15.6 Corporation - xyz@example.com
xy[0,2:word] z[3,4:word] 15[5,7:word] 6[8,9:word] corpor[10,21:word] xyz[24,27:word] exampl[28,35:word]
SmartChineseAnalyzer analyzing : 北京市北京大学
北京市[0,3:word] 北京大学[3,7:word]
```

Analyzer之TokenStream

TokenStream是分析处理组件中的一种中间数据格式，它从一个reader中获取文本，并以TokenStream作为输出结果。在所有的过滤器中，TokenStream同时充当着输入和输出格式。Tokenizer和TokenFilter继承自TokenStream，Tokenizer是一个TokenStream，其输入源是一个Reader；TokenFilter也是一个TokenStream，其输入源是另一个TokenStream。而TokenStream简单点说就是生成器的输出结果。TokenStream是一个分词后的Token结果组成的流，通过流能够不断的得到下一个Token。

```java
@Test
public void testTokenStream() throws IOException {
    Analyzer analyzer = new WhitespaceAnalyzer();
    String inputText = "This is a test text for token!";
    TokenStream tokenStream = analyzer.tokenStream("text", new StringReader(inputText));
    //保存token字符串
    CharTermAttribute charTermAttribute = tokenStream.addAttribute(CharTermAttribute.class);
    //在调用incrementToken()开始消费token之前需要重置stream到一个干净的状态
    tokenStream.reset();
    while (tokenStream.incrementToken()) {
        //打印分词结果
        System.out.print("[" + charTermAttribute + "]");
    }
}
```

输出结果：

> [This][is][a][test][text][for][token!]

Analyzer之TokenAttribute

在调用tokenStream()方法之后，我们可以通过为之添加多个Attribute，从而可以了解到分词之后详细的词元信息，比如CharTermAttribute用于保存词元的内容，TypeAttribute用于保存词元的类型。

在Lucene中提供了几种类型的Attribute，每种类型的Attribute提供一个不同的方面或者token的元数据，罗列如下

名称	作用

名称	作用
CharTermAttribute	表示token本身的内容
PositionIncrementAttribute	表示当前token相对于前一个token的相对位置，也就是相隔的词语数量（例如“text for attribute”，text和attribute之间的getPositionIncrement为2），如果两者之间没有停用词，那么该值被置为默认值1
OffsetAttribute	表示token的首字母和尾字母在原文本中的位置
TypeAttribute	表示token的词汇类型信息，默认值为word，其它值有 <ALPHANUM> <APOSTROPHE> <ACRONYM> <COMPANY> <EMAIL> <HOST> <NUM> <CJ> <ACRONYM_DEP>
FlagsAttribute	与TypeAttribute类似，假设你需要给token添加额外的信息，而且希望该信息可以通过分析链，那么就可以通过flags去传递
PayloadAttribute	在每个索引位置都存储了payload（关键信息），当使用基于Payload的查询时，该信息在评分中非常有用

```
@Test
public void testAttribute() throws IOException {
    Analyzer analyzer = new StandardAnalyzer();
    String input = "This is a test text for attribute! Just add-some word.";
    TokenStream tokenStream = analyzer.tokenStream("text", new StringReader(input));
    CharTermAttribute charTermAttribute = tokenStream.addAttribute(CharTermAttribute.class);
    PositionIncrementAttribute positionIncrementAttribute = tokenStream.addAttribute(PositionIncrementAt
    OffsetAttribute offsetAttribute = tokenStream.addAttribute(OffsetAttribute.class);
    TypeAttribute typeAttribute = tokenStream.addAttribute(TypeAttribute.class);
    PayloadAttribute payloadAttribute = tokenStream.addAttribute(PayloadAttribute.class);
    payloadAttribute.setPayload(new BytesRef("Just"));
    tokenStream.reset();
    while (tokenStream.incrementToken()) {
        System.out.print("[" + charTermAttribute + " increment:" + positionIncrementAttribute.getPositio
                " start:" + offsetAttribute
                .startOffset() + " end:" +
                offsetAttribute
                        .endOffset() + " type:" + typeAttribute.type() + " payload:" + payloadAttribute.
                "]\n");
    }

    tokenStream.end();
    tokenStream.close();
}
```

在调用incrementToken()结束迭代之后，调用end()和close()方法，其中end()可以唤醒当前TokenStream的处理器去做一些收尾工作，close()可以关闭TokenStream和Analyzer去释放在分析过程中使用的资源。

输出结果如下，其中停用词被分词器过滤掉了

> [test increment:4 start:10 end:14 type: payload:null]
> [text increment:1 start:15 end:19 type: payload:null]
> [attribute increment:2 start:24 end:33 type: payload:null]
> [just increment:1 start:35 end:39 type: payload:null]
> [add increment:1 start:40 end:43 type: payload:null]

> [some increment:1 start:44 end:48 type: payload:null]
>
> [word increment:1 start:49 end:53 type: payload:null]

Analyzer之TokenFilter

TokenFilter主要用于TokenStream的过滤操作，用来处理Tokenizer或者上一个TokenFilter处理后的结果，如果是对现有分词器进行扩展或修改，推荐使用自定义TokenFilter方式。自定义TokenFilter需要实现incrementToken()抽象函数，并且该方法需要声明为final的，在此函数中对过滤Term的CharTermAttribute和PositionIncrementAttribute等属性进行操作，就能实现过滤功能，例如一个简单的词扩展过滤器如下

```
import org.apache.lucene.analysis.Analyzer;
import org.apache.lucene.analysis.TokenFilter;
import org.apache.lucene.analysis.TokenStream;
import org.apache.lucene.analysis.core.WhitespaceAnalyzer;
import org.apache.lucene.analysis.tokenattributes.CharTermAttribute;
import org.junit.Test;

import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

public class TestTokenFilter {
    @Test
    public void test() throws IOException {
        String text = "Hi, Dr Wang, Mr Liu asks if you stay with Mrs Liu yesterday!";
        Analyzer analyzer = new WhitespaceAnalyzer();
        CourtesyTitleFilter filter = new CourtesyTitleFilter(analyzer.tokenStream("text", text));
        CharTermAttribute charTermAttribute = filter.addAttribute(CharTermAttribute.class);
        filter.reset();
        while (filter.incrementToken()) {
            System.out.print(charTermAttribute + " ");
        }
    }
}

/**
 * 自定义词扩展过滤器
 */
class CourtesyTitleFilter extends TokenFilter {
    Map<String, String> courtesyTitleMap = new HashMap<>();
    private CharTermAttribute termAttribute;

    /**
     * Construct a token stream filtering the given input.
     *
     * @param input
     */
    protected CourtesyTitleFilter(TokenStream input) {
        super(input);
        termAttribute = addAttribute(CharTermAttribute.class);
        courtesyTitleMap.put("Dr", "doctor");
        courtesyTitleMap.put("Mr", "mister");
        courtesyTitleMap.put("Mrs", "miss");
    }

    @Override
    public final boolean incrementToken() throws IOException {
        if (!input.incrementToken()) {
            return false;
        }
        String small = termAttribute.toString();
        if (courtesyTitleMap.containsKey(small)) {
            termAttribute.setEmpty().append(courtesyTitleMap.get(small));
        }
```

```
        return true;
    }
}
```

输出结果如下

> Hi, doctor Wang, mister Liu asks if you stay with miss Liu yesterday!

自定义Analyzer实现扩展停用词

1. 继承自Analyzer并覆写createComponents(String)方法
2. 维护自己的停用词词典
3. 重写TokenStreamComponents，选择合适的过滤策略

```
class StopAnalyzerExtend extends Analyzer {
    private CharArraySet stopWordSet;//停止词词典

    public CharArraySet getStopWordSet() {
        return this.stopWordSet;
    }

    public void setStopWordSet(CharArraySet stopWordSet) {
        this.stopWordSet = stopWordSet;
    }

    public StopAnalyzerExtend() {
        super();
        setStopWordSet(StopAnalyzer.ENGLISH_STOP_WORDS_SET);
    }

    /**
     * @param stops 需要扩展的停止词
     */
    public StopAnalyzerExtend(List<String> stops) {
        this();
        /**如果直接为stopWordSet赋值的话，会报如下异常，这是因为在StopAnalyzer中有ENGLISH_STOP_WORDS_SET = CharA
         * ENGLISH_STOP_WORDS_SET 被设置为不可更改的set集合
         * Exception in thread "main" java.lang.UnsupportedOperationException
         * at org.apache.lucene.analysis.util.CharArrayMap$UnmodifiableCharArrayMap.put(CharArrayMap.java
         * at org.apache.lucene.analysis.util.CharArraySet.add(CharArraySet.java:105)
         * at java.util.AbstractCollection.addAll(AbstractCollection.java:344)
         * at MyAnalyzer.<init>(AnalyzerDemo.java:146)
         * at MyAnalyzer.main(AnalyzerDemo.java:162)
         */
        //stopWordSet = getStopWordSet();
        stopWordSet = CharArraySet.copy(getStopWordSet());
        stopWordSet.addAll(StopFilter.makeStopSet(stops));
    }

    @Override
    protected TokenStreamComponents createComponents(String fieldName) {
        Tokenizer source = new LowerCaseTokenizer();
        return new TokenStreamComponents(source, new StopFilter(source, stopWordSet));
    }

    public static void main(String[] args) throws IOException {
        ArrayList<String> strings = new ArrayList<String>() {{
            add("小鬼子");
            add("美国佬");
        }};
        Analyzer analyzer = new StopAnalyzerExtend(strings);
        String content = "小鬼子 and 美国佬 are playing together!";
```

```
        TokenStream tokenStream = analyzer.tokenStream("myfield", content);
        tokenStream.reset();
        CharTermAttribute charTermAttribute = tokenStream.addAttribute(CharTermAttribute.class);
        while (tokenStream.incrementToken()) {
            // 已经过滤掉自定义停用词
            // 输出: playing   together
            System.out.println(charTermAttribute.toString());
        }
        tokenStream.end();
        tokenStream.close();
    }
}
```

自定义Analyzer实现字长过滤

```
class LongFilterAnalyzer extends Analyzer {
    private int len;

    public int getLen() {
        return this.len;
    }

    public void setLen(int len) {
        this.len = len;
    }

    public LongFilterAnalyzer() {
        super();
    }

    public LongFilterAnalyzer(int len) {
        super();
        setLen(len);
    }

    @Override
    protected TokenStreamComponents createComponents(String fieldName) {
        final Tokenizer source = new WhitespaceTokenizer();
        //过滤掉长度<len，并且>20的token
        TokenStream tokenStream = new LengthFilter(source, len, 20);
        return new TokenStreamComponents(source, tokenStream);
    }

    public static void main(String[] args) {
        //把长度小于2的过滤掉，开区间
        Analyzer analyzer = new LongFilterAnalyzer(2);
        String words = "I am a java coder! Testingtestingtesting!";
        TokenStream stream = analyzer.tokenStream("myfield", words);
        try {
            stream.reset();
            CharTermAttribute offsetAtt = stream.addAttribute(CharTermAttribute.class);
            while (stream.incrementToken()) {
                System.out.println(offsetAtt.toString());
            }
            stream.end();
            stream.close();
        } catch (IOException e) {
        }
    }
}
```

输出结果如下

```
am
java
coder!
```

可以看到，长度小于两个字符的文本都被过滤掉了。

Analyzer之PerFieldAnalyzerWrapper

PerFieldAnalyzerWrapper的doc注释中提供了详细的说明，该类提供处理不同的Field使用不同的Analyzer的技术方案。PerFieldAnalyzerWrapper可以像其它的Analyzer一样使用，包括索引和查询分析。

```
public void testPerFieldAnalyzerWrapper() throws IOException, ParseException {
    Map<String, Analyzer> fields = new HashMap<>();
    fields.put("partnum", new KeywordAnalyzer());
    //对于其他的域，默认使用SimpleAnalyzer分析器，对于指定的域partnum使用KeywordAnalyzer
    PerFieldAnalyzerWrapper perFieldAnalyzerWrapper = new PerFieldAnalyzerWrapper(new SimpleAnalyzer(),
    Directory directory = new RAMDirectory();
    IndexWriterConfig indexWriterConfig = new IndexWriterConfig(perFieldAnalyzerWrapper);
    IndexWriter indexWriter = new IndexWriter(directory, indexWriterConfig);
    Document document = new Document();
    FieldType fieldType = new FieldType();
    fieldType.setStored(true);
    fieldType.setIndexOptions(IndexOptions.DOCS_AND_FREQS);
    document.add(new Field("partnum", "Q36", fieldType));
    document.add(new Field("description", "Illidium Space Modulator", fieldType));
    indexWriter.addDocument(document);
    indexWriter.close();
    IndexSearcher indexSearcher = new IndexSearcher(DirectoryReader.open(directory));
    //直接使用TermQuery是可以检索到的
    TopDocs search = indexSearcher.search(new TermQuery(new Term("partnum", "Q36")), 10);
    Assert.assertEquals(1, search.totalHits);
    //如果使用QueryParser，那么必须要使用PerFieldAnalyzerWrapper，否则如下所示，是检索不到的
    Query description = new QueryParser("description", new SimpleAnalyzer()).parse("partnum:Q36 AND SPAC
    search = indexSearcher.search(description, 10);
    Assert.assertEquals(0, search.totalHits);
    System.out.println("SimpleAnalyzer :" + description.toString());//+partnum:q +description:space，原因
    //使用PerFieldAnalyzerWrapper可以检索到
    //partnum:Q36 AND SPACE表示在partnum中出现Q36，在description中出现SPACE
    description = new QueryParser("description", perFieldAnalyzerWrapper).parse("partnum:Q36 AND SPACE")
    search = indexSearcher.search(description, 10);
    Assert.assertEquals(1, search.totalHits);
    System.out.println("(SimpleAnalyzer,KeywordAnalyzer) :" + description.toString());//+partnum:Q36 +de
}
```

输出结果如下

```
SimpleAnalyzer :+partnum:q +description:space
(SimpleAnalyzer,KeywordAnalyzer) :+partnum:Q36 +description:space
```

由结果可以看出，在索引阶段，使用KeywordAnalyzer作为partnum域的分词器，使用SimpleAnalyzer作为其它域的分词器，同样的在检索阶段，也必须这样处理，否则无法检索到结果。

坚持原创技术分享，您的支持将鼓励我继续创作！

打赏

本文作者： Wang Xu